

Blazing 8s

Application Web Multijoueur en Temps Réel

Ouaaliti Houssam (L1) Rja-fallah Mohamed Amine (L2) El Amrani Abdelhamid (L2) Abdellaoui Mohamed Khalil (M1)

Applications Concurrentes, Communicantes et Bases de Données — ENSEEIH7, Mai 2026

PRÉSENTATION DU PROJET

Blazing 8s est un jeu de cartes multijoueur en ligne développé en full-stack dans le cadre du cours d'Applications Concurrentes, Communicantes et Bases de Données. Le jeu est inspiré du Blazing 8s : chaque joueur cherche à vider sa main en premier en posant des cartes de même couleur ou de même valeur que celle du dessus de la défausse. Quatre types de cartes spéciales (Wild 8, Oscar's Swap, Giddy Up, Wrong Way) introduisent des rebondissements stratégiques à chaque partie.

L'application couvre toutes les fonctionnalités exigées par l'énoncé : identification et inscription des joueurs, salon d'attente multijoueur, jeu en temps réel synchronisé entre tous les participants, chat intégré pendant la partie, historique complet des coups joués et classement général persisté en base de données. La partie se déroule entre 2 et 4 joueurs, avec une distribution de 5 cartes par joueur au démarrage.

ARCHITECTURE GÉNÉRALE

L'application suit l'architecture MVC imposée, avec une séparation stricte entre un backend Spring Boot et un frontend React. Les deux communiquent via deux canaux complémentaires : REST pour les actions ponctuelles (connexion, lobby, classement) et WebSocket pour tout ce qui nécessite du temps réel (actions de jeu, chat). Le choix du WebSocket est motivé par la nature événementielle du jeu : l'état ne change qu'à chaque action d'un joueur. Le serveur pousse une mise à jour uniquement à ce moment-là, sans sollicitation continue du client.

Stack backend

- Spring Boot
- JPA / Hibernate
- WebSocket
- MySQL

Stack frontend

- React + Vite
- JavaScript
- `api.js` — appels REST
- `websocket.js` — connexions WS

Organisation des packages en backend

<code>entity/</code>	8 entités JPA : <code>Player</code> , <code>Game</code> , <code>GamePlayer</code> , <code>Card</code> , <code>Deck</code> , <code>GameTurn</code> , <code>Message</code> , <code>Ranking</code>
<code>repositories/</code>	Repositories JPA avec requêtes JPQL explicites (<code>@Query</code>) pour les accès ciblés
<code>service/</code>	<code>AuthService</code> · <code>DeckService</code> · <code>GameService</code> · <code>RankingService</code>
<code>controller/</code>	<code>AuthController</code> · <code>LobbyController</code> · <code>GameController</code> · <code>RankingController</code>
<code>dto/</code>	<code>PlayerDTO</code> · <code>CardDTO</code> · <code>GameStateDTO</code> · <code>LobbyGameDTO</code> · <code>RankingDTO</code> · <code>LoginRequest</code> · <code>RegisterRequest</code>
<code>websocket/</code>	<code>GameEndpoint</code> · <code>ChatEndpoint</code> · <code>WebSocketConfig</code>

MODÈLE DE DONNÉES — 8 ENTITÉS JPA

La modélisation a été réalisée lors du Sprint 1. Le schéma compte 8 entités JPA, satisfaisant l'exigence de complexité imposée par l'énoncé.

Entité	Rôle et attributs principaux
<code>Player</code>	Compte joueur : <code>id</code> , <code>username</code> (UNIQUE), <code>password</code>
<code>Game</code>	Partie : <code>status</code> (WAITING / IN_PROGRESS / FINISHED), <code>direction</code> (CLOCKWISE / COUNTERCLOCKWISE), <code>currentColor</code> , <code>currentPlayerId</code> , <code>winner</code>
<code>GamePlayer</code>	Jointure Joueur × Partie enrichie : contient les cartes en main d'un joueur pour une partie donnée, <code>handSize</code> (dénormalisation pour éviter un COUNT à chaque broadcast)
<code>Card</code>	Carte physique : <code>color</code> , <code>value</code> , <code>isDiscard</code> (flag défausse), <code>deckId</code> , <code>holderId</code> (null si dans la pioche)
<code>Deck</code>	Pioche unique par partie (<code>gameId</code>)
<code>GameTurn</code>	Historique d'un coup joué : <code>gameId</code> , <code>playerId</code> , <code>cardId</code>
<code>Message</code>	Message de chat : <code>gameId</code> , <code>senderId</code> , <code>content</code> , <code>sentAt</code>
<code>Ranking</code>	Statistiques d'un joueur : <code>totalGames</code> , <code>totalWins</code> , <code>totalLosses</code>

L'entité **GamePlayer** joue un rôle central : elle lie un joueur à une partie et sert de conteneur pour ses cartes en main. Une carte dont `holderId` est null se trouve dans la pioche ; avec `isDiscard = true`, elle est sur la défausse. Ce schéma à trois états évite toute table supplémentaire.

Les relations JPA sont : `Player` → `GamePlayer` (OneToMany), `Game` → `GamePlayer` (OneToMany), `Game` → `Deck` (OneToOne), `GamePlayer` → `Card` (OneToMany), `Deck` → `Card` (OneToMany), `Player` → `Ranking` (OneToOne).

Le schéma est entièrement défini dans `init.sql` avec `ddl-auto=none`. Ce choix garantit un contrôle total sur les types de colonnes, les contraintes de clés étrangères et les données initiales, sans génération automatique par Hibernate.

LOGIQUE MÉTIER

Composition du deck — 48 cartes

Type	Règle de jeu	Qté
Cartes 1–9 (sauf 8)	Même couleur ou même valeur que la carte du dessus	32
Wild 8	Jouable à tout moment — le joueur choisit la couleur suivante	4
Oscar's Swap	Jouable à tout moment — échange la main avec le joueur suivant	4
Giddy Up	Même couleur que la carte du dessus — bloque le joueur suivant	4
Wrong Way	Même couleur que la carte du dessus — inverse le sens du jeu	4
Total		48

Le `DeckService` crée les 48 cartes, les mélange avec `Collections.shuffle()`, distribue 5 cartes à chaque joueur, puis sélectionne la première carte de la défausse en excluant les Wild 8 et les Oscar's Swap, dont l'effet en tout début de partie serait problématique.

Validation et déroulement d'un tour — `GameService.playCard()`

La méthode `playCard()` est le cœur de la logique. Elle exécute dans l'ordre :

- Vérification du tour** — si `game.getCurrentPlayer().getId()` ne correspond pas au joueur qui envoie l'action, une exception est levée immédiatement. Cela protège contre toute tentative de jouer hors tour, y compris depuis un client modifié.
- Validation du coup** — la méthode `isValidPlay(played, topCard, currentColor)` accepte une carte si : elle partage la couleur active **ou** la valeur du dessus. Wild 8 et Oscar's Swap sont toujours valides. Giddy Up et Wrong Way exigent la couleur active.
- Application de l'effet** selon la valeur de la carte :
 - *Wild 8* → la couleur courante est mise à jour avec le choix du joueur.
 - *Oscar's Swap* → `swapHands()` échange tous les `holder` des cartes des deux joueurs en base.
 - *Giddy Up* → le prochain index est calculé une seconde fois, sautant un joueur.
 - *Wrong Way* → `direction` bascule entre `CLOCKWISE` et `COUNTERCLOCKWISE` ; le prochain joueur est recalculé avec le nouveau sens.
- Calcul du joueur suivant** — `getNextIndex()` applique $(current + 1) \% size$ en `CLOCKWISE` et $(current - 1 + size) \% size$ en `COUNTERCLOCKWISE`.
- Enregistrement dans `GameTurn`** — chaque coup est tracé pour l'historique.
- Détection de fin de partie** — si `cardRepo.findByHolder(winnerGP).isEmpty()`, le statut passe à `FINISHED`, le gagnant est enregistré dans `game.winner`, et `RankingService` appelle `recordWin()` pour le gagnant et `recordLoss()` pour chaque autre participant.

La méthode `drawCard()` pioche la première carte disponible dans la pioche (`cardRepo.findPioche()`), la rattache au `GamePlayer` du joueur, puis calcule et applique immédiatement le joueur suivant — évitant un tour bloqué.

COUCHE DE COMMUNICATION

API REST — 10 endpoints

Méthode	URL	Description
POST	<code>/api/auth/register</code>	Inscription — crée le joueur et son <code>Ranking</code> associé
POST	<code>/api/auth/login</code>	Connexion — retourne le <code>PlayerDTO</code>
GET	<code>/api/lobby</code>	Liste des parties en statut <code>WAITING</code>
POST	<code>/api/lobby/create</code>	Crée une partie et y inscrit le joueur
POST	<code>/api/lobby/join/{id}</code>	Rejoint une partie (max 4 joueurs)
POST	<code>/api/game/{id}/start</code>	Démarre la partie, distribue les cartes, broadcast WS
GET	<code>/api/game/{id}</code>	État courant de la partie pour un joueur donné
POST	<code>/api/game/{id}/play</code>	Joue une carte (<code>cardId</code> , <code>chosenColor</code>)
POST	<code>/api/game/{id}/draw</code>	Pioche et passe le tour
GET	<code>/api/ranking</code>	Classement trié par nombre de victoires

Tous les contrôleurs portent `@CrossOrigin(origins = "*")` pour autoriser les requêtes depuis le frontend React (port

5173).

WebSocket — temps réel

Deux endpoints WebSocket sont actifs pendant une partie :

GameEndpoint (`/ws/game/{gameId}/{playerId}`) reçoit les actions `play` et `draw` en JSON. Après chaque action, la méthode `broadcastGameState()` diffuse un `GameStateDTO` personnalisé à chaque session connectée à la partie. Les erreurs métier (coup invalide, hors tour) sont renvoyées uniquement à la session concernée. Les sessions mortes sont nettoyées à `@OnOpen` et `@OnError` avec `sessions.removeIf(...)` pour éviter les broadcasts sur des sockets fermées.

Lorsque le premier joueur clique sur "Démarrer", `GameController.startGame()` appelle `GameEndpoint.broadcastGameState()` immédiatement après avoir lancé la partie — tous les participants reçoivent leur main via WebSocket sans rafraîchissement.

ChatEndpoint (`/ws/chat/{gameId}/{playerId}`) persiste chaque message dans l'entité `Message` avant de le diffuser, ce qui garantit la conservation du fil de discussion en cas de reconnexion.

GameStateDTO — contenu

Le DTO transmis à chaque joueur après chaque événement contient : statut et direction de la partie, couleur active, **main du joueur connecté uniquement** (les adversaires ne voient pas les cartes des autres), carte du dessus de la défausse, liste des adversaires avec leur `handSize` respectif, joueur dont c'est le tour, et nom du gagnant si la partie est terminée. Ce DTO expose exactement ce dont l'interface a besoin, en évitant la sérialisation des entités JPA complètes (problème de références circulaires Jackson).

FRONTEND REACT

L'interface est une Single Page Application React construite avec Vite, organisée en quatre pages et trois composants réutilisables.

Page / Composant	Rôle
LoginPage	Deux onglets (Connexion / Inscription), gestion des erreurs HTTP retournées par le backend
LobbyPage	Liste des parties avec rafraîchissement auto toutes les 3 s, création et rejoindre une partie
GamePage	Table de jeu complète : main, défausse, pioche, indicateur de couleur active, adversaires, chat, bandeau de fin de partie
RankingPage	Classement trié par victoires avec médailles pour le podium
Card	Composant carte avec couleur, valeur et animation au survol
ColorPicker	Modal de sélection de couleur déclenché uniquement pour le Wild 8
Chat	Panneau de messagerie avec défilement automatique

La session est conservée dans le `localStorage` du navigateur : l'utilisateur reste connecté après un rechargement de page, sans mécanisme de session côté serveur. Les deux services JavaScript `api.js` et `websocket.js` utilisent `window.location.hostname` comme hôte dynamique — l'application est ainsi accessible depuis n'importe quel appareil du réseau local sans modifier le code source.

PROGRESSION PAR SPRINT ET CORRECTIONS

Sprints

Sprint 1 — 01/04	Modélisation complète des 8 entités JPA et de toutes les relations. Mise en place du schéma <code>init.sql</code> et de la configuration MySQL.
Sprint 2 — 07/04	Repositories JPA avec requêtes JPQL. Implémentation des 4 services métier. Logique complète du jeu avec gestion des cartes spéciales. Tests via <code>@PostConstruct</code> .
Sprint 3 — 14/04	Couche DTO. Implémentation des 4 controllers REST. Tests Postman de tous les endpoints. Initialisation du projet React avec Vite.
Sprint 4 — 04/05	Endpoints WebSocket (<code>GameEndpoint</code> , <code>ChatEndpoint</code>). Interface React complète (4 pages, 3 composants). Tests en conditions réelles avec 2 navigateurs.
Final — 19/05	Corrections de tous les bugs identifiés. Support multi-appareils. Stabilisation du schéma SQL sur tous les environnements.

Bugs identifiés et corrigés

Problème	Correction
Pioche sans passage de tour	<code>drawCard()</code> calcule et applique le joueur suivant après la pioche
Message de victoire identique pour tous	Champ <code>winner</code> ajouté dans <code>GameStateDTO</code> , affichage conditionnel vert/rouge côté React
Classement non mis à jour	Appel explicite de <code>RankingService.recordWin()</code> et <code>recordLoss()</code> dans <code>playCard()</code>
Erreur 500 en WAITING (deck inexistant)	Vérification du statut avant tout accès au deck dans <code>getGameState()</code>
Joueur 2 sans cartes au démarrage	<code>GameController.startGame()</code> appelle <code>broadcastGameState()</code> après démarrage
Sessions WebSocket fantômes et doublons	Nettoyage des sessions mortes dans <code>@OnOpen</code> et <code>@OnError</code> via <code>sessions.removeIf(...)</code>
Colonnes <code>sender_id</code> / <code>sent_at</code> absentes	Mise à jour de <code>init.sql</code> et réexécution sur la base MySQL
Inaccessible depuis d'autres appareils	Remplacement des IP en dur par <code>window.location.hostname</code>
Conflit driver HSQLDB / MySQL	Suppression de la dépendance HSQLDB, migration complète vers MySQL

CONCLUSION

Blazing 8s est une application web multijoueur complète répondant à toutes les exigences de l'énoncé : architecture MVC, backend Spring Boot, 8 entités JPA, fonctionnalités multijoueur en temps réel, chat, historique et classement. Développé en 4 sprints successifs, le projet a permis d'appliquer concrètement les concepts du cours : gestion de la concurrence avec des sessions WebSocket multiples, communication temps réel, persistance relationnelle avec JPA, et séparation des responsabilités en couches MVC. Les problèmes rencontrés en phase de test — synchronisation multi-clients, cohérence du schéma SQL entre environnements — ont chacun nécessité une compréhension précise des mécanismes sous-jacents pour être résolus correctement.